Selenium Python AI Agent

Plan. Generate. Heal.



AI agents that write and repair Selenium Python tests from one plain-English sentence — and hand you standard code you own forever.

```
pip install selenium-python-ai-agent
```

Open source (MIT) · Live on PyPI · Built by Ankit Tripathi

The Problem — UI Automation Costs You Twice

Writing it

- Hunting locators, wiring page objects, tuning waits
- Hours of skilled effort for every single flow
- Coverage always lags behind the application

Keeping it alive

- Every UI change breaks selectors somewhere
- Triage and repair eat into every sprint
- Suites rot; teams stop trusting them

Selenium — trusted for two decades

Around since 2004, it remains one of the most widely adopted web-automation standards — and Python is among its largest communities.

The missing piece

Microsoft Playwright ships AI test agents (planner · generator · healer) for its TypeScript ecosystem. Selenium itself ships no equivalent — authoring and maintenance remain fully manual.

This framework is purpose-built to close that gap for the Selenium + Python ecosystem.

The Solution — Describe the Test. Get the Suite.

```
$ selenium-agent "log in as standard_user, add the backpack to
the cart,
  checkout, verify the order confirmation, then logout"
```

▼

What lands in your repo:

- specs/plan.md — reviewable test plan (stakeholder summary + strategy)
- pages/*.py — clean Page Object Model, one class per page
- tests/*.py — pytest or pytest-bdd suite, run and verified green
- Standard Python — reviewed, committed, and owned like any code

1 sentence in

plain English — that is the whole input

~2 min to green

full checkout suite, measured on a live run

0 AI at runtime

plain pytest forever — no per-run API cost

How It Works — Three Agents, Driven by Live Evidence



Planner

- Opens the app in a real browser and scans the live DOM
- Explores linked pages relevant to your instruction
- Saves a reviewable plan: stakeholder summary + test strategy



Generator

- Turns the plan into a full Page Object Model suite
- pytest or pytest-bdd — validated before writing a single file
- Fits a fresh project or your existing framework style



Healer

- Runs the suite; captures failing URL + on-screen errors
- Re-scans that exact page, fixes the code, re-runs
- Green only when tests truly pass — then shows a healing report

Selectors are never guessed — every decision is grounded in what the application actually renders.

Key Capabilities

You own the output

Real code in your repo; runs on plain pytest with zero AI dependency and zero per-run cost.

Project-native mode

Scans your existing framework and generates in its style — your folders, base classes, fixtures.

Verified healing

Run → re-scan live DOM → fix → re-run. A fix only counts when tests actually pass.

Reviewable plans

specs/*.md with a stakeholder summary and test strategy — review before any code exists.

CI-ready CLI

Exit codes, no AI IDE session needed; plus Claude Code integration via init-agents.

Honest limits

CAPTCHA detected and reported as a blocker — never bypassed, never endless retries.

How It Is Different

Each category of tooling solves a different part of the problem:

AI browser tools for IDEs (MCP servers)	Give an AI assistant interactive browser control inside an IDE session. Built for exploration and assisted work — not a standalone pipeline you schedule in CI.
Code assistants (Copilot / ChatGPT)	Draft test code from training knowledge. They do not open your application, so selectors are unverified — and they never run the result.
Runtime AI test agents	An LLM drives or interprets the test on every execution. Flexible, but each run needs the AI — with the cost and dependency that brings.
Self-healing libraries (e.g. Healenium)	Recover from broken locators while the test runs. The repair lives at runtime — the source code itself is not updated.
This framework	Combines the pieces end-to-end: plans from the live DOM, generates pytest / pytest-bdd code you own, verifies by running it, and repairs the source itself when the UI changes. After generation, the suite runs on plain pytest — no AI required.

ROI — One Real Workflow, Measured

Full E2E checkout on saucedemo.com: login → add to cart → checkout form → confirmation → logout (7 page objects + test suite)

Without — hand-written

- Explore the app, identify every locator — 2–3 hrs
- Write 7 page objects — 6–7 hrs
- Write the test, tune waits — 3–4 hrs
- Debug, stabilize, review — 3–4 hrs

≈ 2 days (16+ hrs)

With selenium-python-ai-agent

- One plain-English command
- Plan + generate + run to green — ~2 min (measured, first attempt)
- Engineer reviews the diff — 20–30 min
- LLM cost for the whole suite — a few cents

≈ 30 minutes

90%+ authoring effort saved · *effort figures are typical estimates; generation time measured on a live run*

ROI — Maintenance & Cost Model

UI changed, locator broke — now what?

- Manual: reproduce, inspect DOM, edit, re-run — 2–3 hrs per break
- With healer: selenium-agent --heal-only <test> → evidence-based fix, verified by a passing re-run — 3–5 min
- Healing report shows exactly what broke and what was changed

Team adoption path

1 — New smoke tests via the agent — low risk, instant value

2 — --scan your framework; output matches your conventions

3 — --heal-only in CI triage — one command per break

AI cost: generation only

a few cents per suite — never per test run

$0 per CI run

1,000 nightly runs cost nothing extra — plain pytest

No vendor lock-in

cancel the API key tomorrow; the tests keep running

Try It Today



```
$ pip install selenium-python-ai-agent
$ selenium-agent "test the login page" --url https://your-app.com
```

PyPI: pypi.org/project/selenium-python-ai-agent

Source (MIT): github.com/AnkitDeepBlue/selenium-python-ai-agent

Roadmap: live-browser generation — the agent executes each step while writing the test

Your tests. Your code. Your repo.

Ankit Tripathi